

# P1950R2

## **indirect\_value**: A Free-Store-Allocated Value Type For C++

Published Proposal, 2022-10-15

**This version:**

<https://wg21.link/P1950>

**Issue Tracking:**

[GitHub](#)

**Authors:**

[Jonathan B. Coe](#)

[Antony Peacock](#)

**Audience:**

LEWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Source:**

[https://github.com/jbcoe/indirect\\_value/blob/main/documentation/p1950.md](https://github.com/jbcoe/indirect_value/blob/main/documentation/p1950.md)

---

## Table of Contents

- 1 Abstract**
- 2 Change history**
- 3 Introduction**
  - 3.1 Motivation
    - 3.1.1 Node-based containers
    - 3.1.2 Hot-cold splitting
    - 3.1.3 Pointer to Implementation - PImpl
      - 3.1.3.1 Issues with const-propagation

- 3.1.3.2 Issues with copies
- 3.1.3.3 An `indirect_value` implementation of the PImpl Idiom
- 3.2 Design decisions
  - 3.2.1 Should there be an empty state?
  - 3.2.2 How are allocators supported?
  - 3.2.3 Is there a small-buffer object optimization?
- 3.3 Prior Art
- 3.4 Completeness of T
- 3.5 Comparison with `unique_ptr<T>` and `polymorphic_value<T>`
- 3.6 Impact on the standard

## 4 Technical specifications

- 4.1 Additions in [memory.syn] 20.2.2:
- 4.2 X.X Class template `copier_traits` [copier.traits]
- 4.3 X.Y Class template `default_copy` [default.copy]
- 4.4 X.Z Class template `indirect_value` [indirect\_value]
  - 4.4.1 X.Z.1 Class template `indirect_value` general [indirect\_value.general]
  - 4.4.2 X.Z.2 Class template `indirect_value` synopsis [indirect\_value.synopsis]
  - 4.4.3 X.Z.3 Class template `indirect_value` constructors [indirect\_value.ctor]
  - 4.4.4 X.Z.4 Class template `indirect_value` destructor [indirect\_value.dtor]
  - 4.4.5 X.Z.5 Class template `indirect_value` assignment [indirect\_value.assignment]
  - 4.4.6 X.Z.6 Class template `indirect_value` modifiers [indirect\_value.modifiers]
  - 4.4.7 X.Z.7 Class template `indirect_value` observers [indirect\_value.observers]
  - 4.4.8 X.Z.8 Class template `indirect_value` creation [indirect\_value.creation]

## 5 Acknowledgements

## 6 References

## § 1. Abstract

Add a class template, `indirect_value`, to the C++ Standard Library to support free-store-allocated objects with value-like semantics.

## § 2. Change history

Changes in P1950r2

- Wording fixes.
- Add allocator support.
- `constexpr` support.

Changes in P1950r1

- Add design discussion.
- Add comparison with `unique_ptr` and `polymorphic_value`.
- Add node-based container and hot-cold splitting as further motivational examples.
- Add examples of similar classes already in use.

## § 3. Introduction

The class template, `indirect_value`, confers value-like semantics on a free-store-allocated object. An `indirect_value` may hold an object of a class `T`, copying the `indirect_value` will copy the object `T`. When a parent object contains a member of type `indirect_value<T>` and is accessed through a const access path, `const`ness will propagate from the parent object to the instance of `T` owned by the `indirect_value` member.

### § 3.1. Motivation

It may be desirable for a class member to be an incomplete type or for the storage used by a member object to be separate from the class itself. In both such cases, the member would traditionally be represented as pointer:

```
class MyClass {  
    AType data_member_  
    AnotherType* indirect_data_member_  
};
```

The author of such a class will need to implement special member functions as the compiler-generated special member functions will not function correctly (indirect data will not be copied, assigned to or

deleted, only the pointer).

Special care will be needed when using the class in multithreaded environments. An instance of `MyClass` accessed through a const-access-path will not propagate the `const`-ness to the indirect data (only to the pointer itself) as pointers do not propagate `const` to their pointees.

The class template `indirect_value` is a drop-in replacement for pointer members in cases where non-polymorphic data referenced by the pointer member is logically part of the class. Use of `indirect_value` will ensure that the compiler-generated special member functions behave correctly and that `const` is propagated to indirect data.

### § 3.1.1. Node-based containers

A tree or linked list can be implemented as a node-based container. This gives stable references to data in the nodes at the cost of memory indirection when accessing data.

Nodes would typically contain data and pointer(s) to other nodes so that the list or tree can be navigated:

```
class ForwardListNode {
    int data_;
    ForwardListNode* next_;
};

class ForwardList {
    ForwardListNode* head_;
public:
    // ...
};
```

The special member functions of `ForwardList` would need to be user-implemented as the compiler-generated versions would not copy, move or delete the nodes correctly.

Care must be taken when implementing `const`-qualified member functions as `const` will not propagate down from the `ForwardList` to the `ForwardListNode`s nor to the data that the nodes contain.

Implementing the ForwardList and ForwardListNode with `indirect_value` corrects these issues:

```
class ForwardListNode {
    int data_;
    indirect_value<ForwardListNode> next_;
};

class ForwardList {
    indirect_value<ForwardListNode> head_;
public:
    // ...
};
```

Compiler-generated special member functions will behave correctly and do not need to be written manually. `const`-propagation will allow the compiler to ensure that data that is logically part of the container cannot be modified through a `const` access path.

### § 3.1.2. Hot-cold splitting

When working with collections of data, CPU caches are often under-utilized when algorithms access certain data members much more frequently than others. This results in some cache lines becoming busier than others, prefetchers preemptively caching memory which is later evicted without being used by the CPU or memory bandwidth becoming a limiting factor in performance. In such cases segregating data structures in terms of hot and cold data can remove pressure on system resources.

```
class Element {
    SmallData frequently_accessed_data;
    LargeData infrequently_accessed_data;
};

vector<Element> elements;
auto active = find_if(elements.begin(),
                      elements.end(),
                      [](const auto& e)
                      {
                          return e.frequently_accessed_data.active();
                      });
```

In such cases adding a level of indirection to the larger, less frequently-accessed data can relieve bandwidth pressure [C. Ericson]. Using `indirect_value` in this refactoring does not change guarantees around `const`ness. Compiler-generated special member functions will behave correctly and do not need to be manually implemented.

```
class Element {
    SmallData frequently_accessed_data;
    indirect_value<LargeData> infrequently_accessed_data;
};

vector<Element> elements;
auto active = find_if(elements.begin(),
                      elements.end(),
                      [](const auto& e)
                      {
                          return e.frequently_accessed_data.active();
                      });
```

### § 3.1.3. Pointer to Implementation - PImpl

In C++, when anything in a class definition changes, dependent classes require recompilation. As early as 1992 the Handle/Body idiom was suggested to break this dependency [J. Coplien]. From this pattern, the PImpl idiom was specialised [H. Sutter]. Despite its relative maturity, using the PImpl idiom requires careful thought about copying and `const`-propagation.

```
// Header file
class widget {
public:
    widget();
    ~widget();
private:
    class impl;
    std::unique_ptr<impl> pimpl_;
};
```

```
// Implementation file
class widget::impl {
    // :::
};
```

```

widget::widget() : pimpl_{ std::make_unique<impl>( /*...*/ } { }
// Destructor needs to be defined in a the same TU as <code data-opaque bs-
widget::~~widget() = default;

```

For convenience, the widget class will be referred to as the “visible class” and impl class the “PImpl class”. Note, semantically the PImpl class is the implementation details of the visible class.

#### § 3.1.3.1. Issues with const-propagation

Using `std::unique_ptr` to store the implementation object introduces an issue - within a `const`-qualified member function, an instance of the visible class can mutate data inside the implementation object. This is because `const`-qualification applies only to the `unique_ptr` value, and not the pointed-to-object.

The compiler is unable to make thread-compatibility assumptions for `const` objects when `const` does not propagate: `const` does not mean immutable in the face of pointer-like-member data.

The desired semantics of a PImpl-owner are value-like, like those of `std::optional` which has appropriate `const` and non-`const`-qualified overloads for `operator*` and `operator->`.

#### § 3.1.3.2. Issues with copies

The copy-constructor and copy-assignment operator of `std::unique_ptr` are deleted. Users of a class with a `std::unique_ptr` member will be required to implement copy-construction and copy-assignment [S. Meyers].

#### § 3.1.3.3. An `indirect_value` implementation of the PImpl Idiom

Using `indirect_value` to implement the PImpl idiom ensures that the PImpl object is `const` when accessed through a `const` access path and that compiler-generated special member functions will behave correctly and do not need to be manually implemented.

```

// Header file
class widget {
public:
    widget();

```

```

    widget(widget&& rhs) noexcept;
    widget(const widget& rhs);
    widget& operator=(widget&& rhs) noexcept;
    widget& operator=(const widget& rhs);
    ~widget();
private:
    class impl;
    std::indirect_value<impl>> pimpl;
};

```

```

// Implementation file
class widget::impl {
    // :::
};

// Special-member functions need to be defined in a the same TU as <code da
widget::widget() : pimpl(new impl) {}
widget::widget(widget&& rhs) noexcept = default;
widget::widget(const widget& rhs) = default;
widget& widget::operator=(widget&& rhs) noexcept = default;
widget& widget::operator=(const widget& rhs) = default;
widget::~~widget() = default;

```

## § 3.2. Design decisions

### § 3.2.1. Should there be an empty state?

There is a design decision to be made about the default constructor of `indirect_value`. It could place `indirect_value` in an empty state with no owned object or it could default construct an owned object so that `indirect_value` is never empty. This would either add a requirement that `T` supports default construction or only support default construction conditionally when also supported by `T`.

If `indirect_value<T>` is never empty then `T` must be default constructible for `indirect_value<T>` to be a regular type. The default constructor of `indirect_value<T>` would then need to allocate memory, which cannot be controlled by the user-supplied copier or deleter.

If the default constructed state of `indirect_value` is non-empty then the moved from state should be similarly non-empty. This requires that `T` is noexcept move constructible.

Allowing an empty state for `indirect_value<T>` imposes preconditions on `operator*` and `operator->` (similar to those of the raw pointer it may be replacing) but removes requirements on `T` and the need to allocate memory.

A nullable `indirect_value<T>` could be mimicked with `std::optional<indirect_value<T>>` but this is verbose, does not solve the problem of `T` needing to be noexcept move-constructible and would require partial template specialization of `std::optional` to avoid the overhead that would be incurred by a nullable `indirect_value`. But perhaps the most unfortunate side effect of this approach is that to access the underlying `T` the `std::optional` must be dereferenced to get to the `indirect_value`, which then requires a second dereference to get to the underlying `T`. This pattern makes usage unwieldy.

As designed, `indirect_value` has an empty state where it owns no object. A default constructed `indirect_value` is empty and the empty state is the state of a moved-from object.

### § 3.2.2. How are allocators supported?

It may be desirable for users to control the way memory is managed by an instance of a class. Classes such as `std::vector` and `std::string` allow an allocator to be specified as a template argument to control memory management. There is existing precedence for allocator support of types in the memory management library; `std::shared_ptr` supports allocators via `std::allocate_shared`, [M. Knejp] suggested the introduction of `std::allocate_unique` to support allocator use with `std::unique_ptr`.

Like `std::unique_ptr`, `indirect_value` allows a deleter to be provided as a template argument so that disposal of the owned object can be customized. `indirect_value` also allows copying to be customized by specifying a copier, another template argument. However, this raises the question of if the copier and deleter should be combined types. If not combined then both must hold either a reference to an allocator (doubling the storage requirements) or, if encapsulating the allocator, then one must own the allocator while the second holds a reference. While combining the interfaces into one type would solve this it would reduce the usage with custom lambdas without specialising for a combined object to be created from the two copier and deleter lambda via something akin to the overload pattern. This issue is not addressed further, instead opting to keep to the pre-allocator design, but the authors welcome feedback here.

Memory management for `indirect_value` can be fully controlled via the `allocate_indirect_value` function which allows passing in an allocator to control the source of memory. Custom copier and deleter then use the allocator for allocations and deallocations.

### § 3.2.3. Is there a small-buffer object optimization?

Previous revisions of the paper suggested this was permissible as a quality of implementation improvement. However, this revision has introduced allocator support, which offers an alternative mechanism for supporting the small-buffer optimization. An allocator, supporting small buffer optimization, can be provided. This allows achieving the same results while separating the details of the small buffer optimization from `indirect_value`.

### § 3.3. Prior Art

There have been previous proposal for deep-copying smart pointers that proposed copy semantics [W. Brown]. `cloned_ptr` was proposed in [J. Coe], however under guidance of LEWG this was renamed to `polymorphic_value`. With this change in name came the addition of const propagation.

This paper is not unique in these ideas. GitHub code search finds 602k lines of code referencing "Pimpl" and 99 C++ repositories claiming to provide generic implementations of Pimpl. Google Carbon Language, an experimental successor to C++ includes `IndirectValue` as a vocabulary type. Additionally other authors have addressed this topic [A. Upadyshev]. Some generic implementations of note in the wild are:

| Project         | Link                                                                                                                                                                              | Deep Copying | Const Propagation |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------------|
| Carbon Language | <a href="https://github.com/carbon-language/carbon-lang/blob/trunk/common/indirect_value.h">https://github.com/carbon-language/carbon-lang/blob/trunk/common/indirect_value.h</a> | Yes          | Yes               |
| Boost Pimpl     | <a href="https://github.com/sean-/boost-pimpl">https://github.com/sean-/boost-pimpl</a>                                                                                           | Yes          | Yes               |
| pimpl           | <a href="https://github.com/JonChesterfield/pimpl">https://github.com/JonChesterfield/pimpl</a>                                                                                   | Yes          | No                |
| impl_ptr        | <a href="https://github.com/sth/impl_ptr">https://github.com/sth/impl_ptr</a>                                                                                                     | Yes          | Yes               |
| Simpl           | <a href="https://github.com/oliora/samples/blob/master/spimpl.h">https://github.com/oliora/samples/blob/master/spimpl.h</a>                                                       | Yes          | Yes               |
| smart_pimpl     | <a href="https://github.com/SBreezyx/smart_pimpl">https://github.com/SBreezyx/smart_pimpl</a>                                                                                     | No           | No                |
| pimpl_on_stack  | <a href="https://github.com/kuznetsss/pimpl_on_stack/">https://github.com/kuznetsss/pimpl_on_stack/</a>                                                                           | Yes          | Yes               |
| deep_const_ptr  | <a href="https://github.com/torbjoernk/deep_const_ptr">https://github.com/torbjoernk/deep_const_ptr</a>                                                                           | No           | Yes               |

Divergence on issues such as deep copying and const propagation along with subtleties in implementation mean that a single standardized solution would be of broad benefit to the community of C++ users.

### § 3.4. Completeness of T

Smart pointer types in the Standard Library expect that some of the members can be instantiated with incomplete types [H.Hinnant]. Similarly, this is the case for `indirect_value`, the table outlines the expected behaviour for incomplete pointer types:

| Function                                                                                  | Description                | Incomplete/Complete |
|-------------------------------------------------------------------------------------------|----------------------------|---------------------|
| <code>indirect_value()</code>                                                             | Default constructor        | Incomplete          |
| <code>indirect_value(const indirect_value&amp;)</code>                                    | Copy-constructor           | Complete            |
| <code>indirect_value(indirect_value&amp;&amp;)</code>                                     | Move-constructor           | Incomplete          |
| <code>~indirect_value()</code>                                                            | Destructor                 | Complete            |
| <code>indirect_value&amp;<br/>indirect_value::operator=(const indirect_value&amp;)</code> | Copy-assignment            | Complete            |
| <code>indirect_value&amp;<br/>indirect_value::operator=(indirect_value&amp;&amp;)</code>  | Move-assignment            | Complete            |
| <code>T&amp; operator*()</code>                                                           | Indirection-operator       | Incomplete          |
| <code>const T&amp; operator*() const</code>                                               | Indirection-operator       | Incomplete          |
| <code>T* operator-&gt;() noexcept</code>                                                  | Member-of-pointer-operator | Incomplete          |
| <code>const T* operator-&gt;() const noexcept</code>                                      | Member-of-pointer-operator | Incomplete          |
| <code>explicit operator bool() const noexcept</code>                                      | Bool-operator              | Incomplete          |
| <code>void swap(indirect_value&amp; p) noexcept</code>                                    | Swap                       | Incomplete          |

### § 3.5. Comparison with `unique_ptr<T>` and `polymorphic_value<T>`

The class template `indirect_value<T>` is in many respects similar to `unique_ptr<T>` and the proposed `polymorphic_value<T>` [J. Coe]. The table below highlights the similarities and differences:

| Behaviour                 | <code>indirect_value</code> | <code>polymorphic_value</code> | <code>unique_ptr</code> |
|---------------------------|-----------------------------|--------------------------------|-------------------------|
| Default constructed state | Empty                       | Empty                          | Empty                   |

| Behaviour                  | <code>indirect_value</code> | <code>polymorphic_value</code> | <code>unique_ptr</code> |
|----------------------------|-----------------------------|--------------------------------|-------------------------|
| Copyable                   | Yes                         | Yes                            | No                      |
| Const-propagating          | Yes                         | Yes                            | No                      |
| Polymorphic                | No                          | Yes                            | Yes                     |
| Customizable memory access | Yes                         | No                             | Yes                     |

### § 3.6. Impact on the standard

This proposal is a pure library extension. It requires additions to be made to the standard library header `<memory>`.

## § 4. Technical specifications

### § 4.1. Additions in [memory.syn] 20.2.2:

```
// [indirect.value], class template indirect_value
template <class T> struct default_copy;
template <class T> struct copier_traits;

template <class T, class C = std::default_copy<T>, class D = typename std::
...
template <class T, class ...Ts>
    constexpr indirect_value<T> make_indirect_value(Ts&& ...ts);

template <class T, class A = std::allocator<T>, class... Ts>
    constexpr indirect_value<T> allocate_indirect_value(std::allocator_arg_t,
...
template<class T>
    constexpr void swap(indirect_value<T>& p, indirect_value<T>& u) noexcept;
```

### § 4.2. X.X Class template `copier_traits` [copier.traits]

```
namespace std {
    template <class T>
        struct copier_traits {
```

```

        using deleter_type = *see below*;
    };
}

```

```
using deleter_type = see below;
```

- Type: `T::deleter_type` if the *qualified-id* `T::deleter_type` is valid and denotes a type; otherwise, `void (*) (U*)` if `T` is of the form `U* (*) (V)` for types `U` and `V`; otherwise, there is no member `deleter_type`.

### § 4.3. X.Y Class template `default_copy` [`default.copy`]

```

namespace std {
    template <class T>
    struct default_copy {
        using deleter_type = default_delete<T>;
        constexpr T* operator()(const T& t) const;
    };
} // namespace std

```

The class template `default_copy` [J. Coe] serves as the default copier for the class template `indirect_value`. The template parameter `T` of `default_copy` may be an incomplete type.

```
constexpr T* operator()(const T& t) const;
```

- Returns: `new T(t);`

### § 4.4. X.Z Class template `indirect_value` [`indirect_value`]

#### § 4.4.1. X.Z.1 Class template `indirect_value` general [`indirect_value.general`]

An `indirect_value` is an object that owns another object and manages that other object through a pointer. More precisely, an indirect value is an object `v` that stores a pointer to a second object `p` and will dispose of `p` when `v` is itself destroyed (e.g., when leaving block scope (9.7)). In this context, `v` is said to own `p`.

An `indirect_value` object is empty if it does not own a pointer.

Copying a non-empty `indirect_value` will copy the owned object so that the copied `indirect_value` will have its own unique copy of the owned object.

Copying from an empty `indirect_value` produces another empty `indirect_value`.

Copying and disposal of the owned object can be customised by supplying a copier and deleter.

The template parameter `T` of `indirect_value` must be a non-union class type.

The template parameter `T` of `indirect_value` may be an incomplete type.

A copier and deleter are said to be present if a `indirect_value` object is constructed from a non-null pointer, or from a `indirect_value` object where a copier and deleter are present.

#### § 4.4.2. X.Z.2 Class template `indirect_value` synopsis [`indirect_value.synopsis`]

```
template <class T, class C = std::default_copy<T>, class D = typename std::default_deleter<T>>
class indirect_value {
public:
    using value_type = T;

    // Constructors
    constexpr indirect_value() noexcept;
    constexpr explicit indirect_value(T* p, C c=C{}, D d=D{});

    constexpr indirect_value(const indirect_value& p);
    constexpr indirect_value(indirect_value&& p) noexcept;

    template <class ...Ts>
    constexpr indirect_value(std::in_place_t, Ts&&... ts); // See below

    // Destructor
    constexpr ~indirect_value();

    // Assignment
    constexpr indirect_value& operator=(const indirect_value& p);
    constexpr indirect_value& operator=(indirect_value&& p) noexcept;

    // Modifiers
    constexpr void swap(indirect_value<T>& p) noexcept;

    // Observers
    constexpr T& operator*();
    constexpr T* operator->();
    constexpr const T& operator*() const;
```

```

        constexpr const T* operator->() const;
        constexpr explicit operator bool() const noexcept;
    };

    // indirect_value creation
    template <class T, class ...Ts>
    constexpr indirect_value<T> make_indirect_value(Ts&& ...ts); // See be

    template <class T, class A = std::allocator<T>, class... Ts>
    constexpr indirect_value<T> allocate_indirect_value(std::allocator_arg_

    // indirect_value specialized algorithms
    template<class T>
    constexpr void swap(indirect_value<T>& p, indirect_value<T>& u) noexcept

} // end namespace std

```

#### § 4.4.3. X.Z.3 Class template `indirect_value` constructors [`indirect_value.ctor`]

```
constexpr indirect_value() noexcept;
```

- Effects: Constructs an empty `indirect_value`.
- Postconditions: `bool(*this) == false`.

```
constexpr explicit indirect_value(T* p, C c=C{}, D d=D{});
```

- Effects: Constructs a `indirect_value` which owns `p`, initializing the stored pointer with `p`. The copier and deleter of the `indirect_value` constructed are moved from `c` and `d`. If `p` is null, creates an empty object.
- Constraints: `is_nothrow_move_assignable_v<C>` is true.  
`is_nothrow_move_assignable_v<D>` is true.
- Preconditions: `C` and `D` meet the *Cpp17CopyConstructible* requirements. If the arguments `c` and/or `d` are not supplied, then `C` and/or `D` respectively are default constructible types that are not pointer types. If `p` is non-null then the expression `c(*p)` returns a non-null `T*` is as if copy constructed from `*p`. The expression `d(p)` has well-defined behaviour, and does not throw exceptions.
- Postconditions: `bool(*this) == bool(p)`.
- Mandates: The expression `d(p)` is well-formed.

```
constexpr indirect_value(const indirect_value& p);
```

- Constraints: `is_copy_constructible_v<T>` is true.
- Effects: Creates a `indirect_value` object that owns a copy of the object managed by `p`. The copy is created by the copier in `p`. If `p` has a custom copier and deleter then the custom copier and deleter of the `indirect_value` constructed are copied from those in `p`.
- Remarks: The constructor must work with pointer to an complete type `T`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Postconditions: `bool(*this) == bool(p)`.

```
constexpr indirect_value(indirect_value&& p) noexcept;
```

- Effects: Move-constructs an `indirect_value` instance from `p`. If `p` has a custom copier and deleter then the copier and deleter of the `indirect_value` constructed are the same as those in `p`.
- Postconditions: `*this` contains the old value of `p`. `p` is empty.

```
constexpr indirect_value(std::in_place_t, Ts&& ...ts);
```

- Effects: Constructs an `indirect_value` which owns an object of type `T` direct-non-list-initialized with `std::forward<Ts>(ts)...`
- Throws: Any exception thrown by the selected constructor of `T` or `bad_alloc` if required storage cannot be obtained.
- Preconditions: `is_same_v<C, default_copy> && is_same_v<D, default_delete>`.

#### § 4.4.4. X.Z.4 Class template `indirect_value` destructor [`indirect_value.dtor`]

```
constexpr ~indirect_value();
```

- Effects: If `*this` there are no effects. If a custom deleter `d` is present then `d(p)` is called and the copier and deleter are destroyed. Otherwise the destructor of the managed object is called.
- Remarks: The constructor must work with pointer to an complete type `T`.

#### § 4.4.5. X.Z.5 Class template `indirect_value` assignment [`indirect_value.assignment`]

```
constexpr indirect_value& operator=(const indirect_value& p);
```

- Constraints: `is_copy_assignable_v<C>` is true. `is_copy_assignable_v<D>` is true.
- Effects: If `p` is empty, assigns an empty object. Otherwise creates an object that owns a copy of the object managed by `p`. The copy is created by the copier in `p`, and the copier and deleter of `*this` are copied from those in `p`.
- Remarks: The function must work with pointer to an complete type `T`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Returns: `*this`.
- Postconditions: `bool(*this) == bool(p)`.

```
constexpr indirect_value& operator=(indirect_value&& p) noexcept;
```

- Constraints: `is_nothrow_move_assignable_v<C>` is true.  
`is_nothrow_move_assignable_v<D>` is true.
- Effects: Ownership of the resource managed by `p` is transferred to this. If `p` has a custom copier and deleter then the copier and deleter of `*this` is the same as those in `p`.
- Remarks: The function must work with pointer to an complete type `T`.
- Returns: `*this`.
- Postconditions: `*this` contains the old value of `p`. `p` is empty.

#### § 4.4.6. X.Z.6 Class template `indirect_value` modifiers [`indirect_value.modifiers`]

```
constexpr void swap(indirect_value& p) noexcept;
```

- Effects: Exchanges the contents of `p` and `*this`.

#### § 4.4.7. X.Z.7 Class template `indirect_value` observers [`indirect_value.observers`]

```
constexpr T& operator*();  
constexpr const T& operator*() const;
```

- Preconditions: `bool(*this)` is true.

- Returns: A reference to the owned object.

```
constexpr T* operator->() noexcept;
constexpr const T* operator->() const noexcept;
```

- Preconditions: `bool(*this)` is true.
- Returns: A pointer to the owned object.

```
constexpr explicit operator bool() const noexcept;
```

- Returns: false if the `indirect_value` is empty, otherwise true.

#### § 4.4.8. X.Z.8 Class template `indirect_value` creation [indirect\_value.creation]

```
template <class T, class U=T, class ...Ts>
constexpr indirect_value<T> make_indirect_value(Ts&& ...ts);
```

- Constraints: `is_constructible_v<U, Ts...>` is true.
- Preconditions: `U` meets the `Cpp17CopyConstructible` requirements.
- Returns: A `indirect_value<T>` owning an object of type direct-non-list-initialized with `std::forward<Ts>(ts)...`.
- Preconditions: `is_same_v<C, default_copy> && is_same_v<D, default_delete>`.

```
template <class T, class U = T, class A = std::allocator<U>, class... Ts>
constexpr indirect_value<T> allocate_indirect_value(std::allocator_arg_t, A
```

- Constraints: `is_constructible_v<U, Ts...>` is true.
- Preconditions: `U` meets the `Cpp17CopyConstructible` requirements.
- Returns: A `indirect_value<T>` owning an object of type direct-non-list-initialized with `std::forward<Ts>(ts)...`.
- Preconditions `is_same_v<C, allocator_copy> && is_same_v<D, allocator_delete>`.

## § 5. Acknowledgements

Contributions to the open source reference implementation have forwarded the design, including but not limited to contribution from Kilian Henneberger, Ed Catmur, Stephen Kelly & Malcolm Parsons. The authors would like to thank Thomas Russell for the original suggestions of allocator support, Bengt Gustafsson for useful discussion on SBO and nullstate representation via `std::optional` specialisation, Jonathan Wakely & Daniel Krügler for document reviews, Andrew Bennieston for useful discussions on the topic and the BSI panel for on-going support.

## § 6. References

[W. Brown] [n3339: A Preliminary Proposal for a Deep-Copying Smart Pointer](#), Walter E. Brown, 2012

[J. Coe] [p0201r5: A polymorphic value-type for C++](#)

[J. Coplien] Advanced C++ Programming Styles and Idioms (Addison-Wesley), James O. Coplien, 1992

[C. Ericson] [Memory Optimization](#), Christer Ericson, Games Developers Conference, 2003

[R. Grimm] [Visiting a `std::variant` with the Overload Pattern](#)

[H. Hinnant] [“Incomplete types and `shared\_ptr` / `unique\_ptr`”](#), Howard Hinnant, 2011

[M. Knejp] [P0316R0: `allocate\_unique` and `allocator\_delete`](#)

[H. Sutter] ["Pimpls - Beauty Marks You Can Depend On"](#), Herb Sutter, 1998

[Impl] [Reference implementation: `indirect\_value`](#), J.B.Coe

[S. Meyers] Effective Modern C++, Item 22: When using the Pimpl Idiom, define special member functions in the implementation file, Scott Meyers, 2014

[A. Upadyshev] [PIMPL, Rule of Zero and Scott Meyers](#), Andrey Upadyshev, 2015